

Mycellius — Séance 2 (4h) : Sprint 1

Socle + workflow dev/test/prod + CI « vide mais vert » — guide pas à pas (avec commandes)

Ce qu'on a fait dans la séance précédente:

- un repo GitHub structuré + branches dev/test/prod
- 3 VM Proxmox (dev/test/prod) prêtes (SSH + sécurité minimale)
- Docker + nginx installés
- une preuve de déploiement : une page statique différente selon l'environnement (DEV/TEST/PROD) servie sur chaque VM
- une CI GitHub Actions minimale qui passe (même si l'application n'existe pas encore)
- une baseline sécurité (CIA + mini registre des risques)

Ce qu'on va faire dans cette séance 2 :

- poser les bases de programmation Java,
- introduire la POO sur des exemples simples,
- commencer à modéliser Mycellius (Tag, WikiPage, InMemoryWiki),
- et connecter tout ça au workflow Git (dev → test).

Découpage de la séance

1. Installation de l'environnement Java
2. Bases de programmation Java : types, variables, opérations, contrôle (45–60 min)
3. POO de base : classe, objet, attribut, méthode (≈ 45 min)
4. POO appliquée à Mycellius : Tag & WikiPage (≈ 45–60 min)
5. JUnit sur règles simples (≈ 30–40 min)
6. CI compile + tests
7. Workflow dev -> test

1. Installation de l'environnement Java

(JDK + IntelliJ + lien avec VS Code & Git)

1.1 - Comment VS Code et IntelliJ vont travailler ensemble ?

Avant de cliquer partout, il faut comprendre l'ensemble :

- Tu as déjà un dépôt Git mycellius/ sur ton PC (avec docs, infra, etc.).

VsCode, tu t'en sers pour :

- éditer les fichiers (README, docs, infra/...),
- faire tes git add, git commit, git push,
- gérer tes PR vers GitHub.

=> IntelliJ ne va PAS remplacer VS Code. Il va juste être ton atelier spécialisé "Java" pour ce même dossier.

En gros, ton PC contient :

- ton dossier mycellius/ unique (avec .git dedans).
- 2 IDE :
 - > VsCode → pour Git, markdown, infra, scripts...
 - > IntelliJ → pour éditer le code Java dans api/.

Git ne dépend pas de l'IDE.

Que tu modifies un fichier Java dans VsCode ou IntelliJ, ça reste le même fichier dans mycellius/api/....

Les PR continueront exactement comme avant :

> tu peux faire les commits soit dans VS Code, soit dans IntelliJ, soit en ligne de commande.

Mais petite recommandation :

=> Garde Git et les PR dans VS Code, et utilise IntelliJ juste pour coder en Java.

Comme ça, tu ne changes qu'un seul "rituel Git".

1.2 - Vérifier (ou installer) le JDK

On va d'abord vérifier si Java est installé :

> **JDK = Java Development Kit** contient :

- la JVM (pour exécuter) → commande java
- le compilateur (pour transformer .java en bytecode) → commande javac

Par analogie :

> **java** = le robot qui exécute le programme déjà compilé.

> **javac** = l'ouvrier qui transforme ton code .java en bytecode.

👉 Sans javac, tu peux lancer des programmes Java déjà compilés, mais tu ne peux pas réellement développer.

Ouvre un terminal sur ton poste Windows, soit PowerShell, soit le terminal intégré de VS Code et tape d'abord :

```
java -version
```

Si tu vois quelque chose comme java version "17...." ou 21, c'est bon :

→ ça veut dire que la JVM (exécution) est installée.

Ensuite tape :

```
javac -version
```

=> Si tu vois une version (javac 17... ou 21...) → tu as bien le JDK (compilateur).

=> Si tu as un message du genre javac : commande introuvable, ça veut dire : pas de JDK → il faudra l'installer.

```
PS C:\Users\clakg\mycellius> java -version
java version "1.8.0_471"
Java(TM) SE Runtime Environment (build 1.8.0_471-b09)
Java HotSpot(TM) 64-Bit Server VM (build 25.471-b09, mixed mode)
PS C:\Users\clakg\mycellius> javac -version
javac: The term 'javac' is not recognized as a name of a cmdlet, function, script file, or executable program.
Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
```

Je vais partir du principe que je n'ai rien pour travailler en Java.

On va donc installer proprement un JDK avant de toucher à IntelliJ ou au moindre code.

a- Installer un JDK moderne LTS (17 ou 21 → je te propose 21).

b- Vérifier que java et javac fonctionnent.

a- Installer un JDK moderne LTS (17 ou 21 → je te propose 21)

On va télécharger un JDK 21 windows 64 bits :

<https://www.oracle.com/fr/java/technologies/downloads/#jdk21-windows>

Lance l'installation du .exe

L'assistant d'installation va s'ouvrir :

Java 25, Java 21, and earlier versions available now

JDK 25 is the latest Long-Term Support (LTS) release of the Java SE Platform.
 JDK 21 is the previous Long-Term Support (LTS) release of the Java SE Platform.
 Earlier JDK versions are available below.

JDK 25 JDK 21

Java SE Development Kit 21.0.9 downloads

JDK 21 binaries are free to use in production and free to redistribute, at no cost, under the [Oracle No-Fee Terms and Conditions \(NFTC\)](#).
 JDK 21 will receive updates under the NFTC, until September 2026, a year after the release of the next LTS. Subsequent JDK 21 updates will be licensed under the [Java SE OTN License \(OTN\)](#) and production use beyond the [limited free grants](#) of the OTN license will [require a fee](#).

Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	186.13 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	164.45 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	163.20 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

- clique sur Next / Suivant,
- tu peux garder les chemins par défaut,
- si une case propose "Set JAVA_HOME" ou "Ajouter au PATH", coche-la → ça t'évitera de la config manuellement.
- Clique sur Install / Installer. Puis, à la fin, clique sur Finish / Terminer.

Si ça ne fonctionne pas, ça voudrait dire que le chemin n'est pas reconnu, et il va donc falloir le configurer à la main.

Dans powershell, en admin, on crée la variable d'environnement JAVA_HOME pour ton compte Windows :

```
$env:JAVA_HOME = "C:\Program Files\Java\jdk-21"
[Environment]::SetEnvironmentVariable("JAVA_HOME", $env:JAVA_HOME, "User")
```

Puis on ajoute le bin du JDK dans le PATH utilisateur :

```
$oldPath = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path", "$oldPath;$env:JAVA_HOME\bin", "User")
```

En gros, ça dit : "prends l'ancien PATH utilisateur, et ajoute C:\Program Files\Java\jdk-21\bin à la fin".

b- Vérifier que java et javac fonctionnent.

Toujours dans ton terminal PS :

> java -version

> javac -version

```
PS C:\WINDOWS\system32> $env:JAVA_HOME = "C:\Program Files\Java\jdk-21"
PS C:\WINDOWS\system32> [Environment]::SetEnvironmentVariable("JAVA_HOME", $env:JAVA_HOME, "User")
PS C:\WINDOWS\system32> $oldPath = [Environment]::GetEnvironmentVariable("Path", "User")
PS C:\WINDOWS\system32> [Environment]::SetEnvironmentVariable("Path", "$oldPath;$env:JAVA_HOME\bin", "User")
PS C:\WINDOWS\system32> java -version
java version "21.0.9" 2025-10-21 LTS
Java(TM) SE Runtime Environment (build 21.0.9+7-LTS-338)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.9+7-LTS-338, mixed mode, sharing)
PS C:\WINDOWS\system32> javac -version
javac 21.0.9
```

1.3 - Installation et configuration IntelliJ

Pour configurer IntelliJ pour utiliser ce JDK il faudra :

- Installer / ouvrir IntelliJ IDEA Community
- Ouvrir ton dépôt mycellius dans IntelliJ (sans casser Git)
- Créer le module Java api (Maven)
- Créer une classe App avec une méthode main et l'exécuter

À la fin, tu auras :

- un "Hello Mycellius" qui s'exécute depuis IntelliJ,
- et ton repo Git reste géré par VS Code comme avant.

Mais avant de faire tout ça, je remets dans le contexte juste pour bien poser les rôles :

VS Code, tu continues à l'utiliser pour :

- ouvrir le dossier C:\Users\clakg\mycellius
- faire git status, git add, git commit, git push
- modifier les fichiers docs, infra, .github, etc.

=> C'est ton outil Git + "généraliste".

IntelliJ IDEA, tu vas l'utiliser pour :

- coder en Java dans mycellius/api
- profiter de l'auto-complétion, des refactorings, etc.

=> C'est ton atelier Java.

Les deux IDE regardent le même dossier mycellius/ sur ton disque.

Donc :

- un fichier modifié dans IntelliJ → VS Code le verra aussi.
- un git commit fait dans VS Code (ou en ligne de commande) → IntelliJ voit le changement.

=> Git reste unique (le dossier .git à la racine), les deux éditeurs ne sont que des “fenêtres” sur les mêmes fichiers.

Allez, on continue...

a- Installer / ouvrir IntelliJ IDEA Community

b- Ouvrir ton dépôt mycellius dans IntelliJ (sans casser Git)

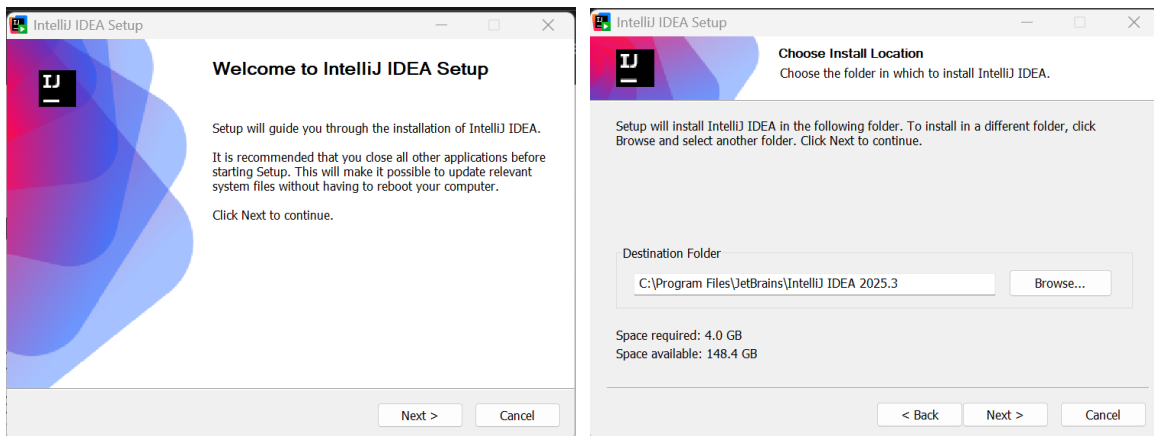
c- Créer le module Java api (Maven)

d- Découvrir JAVA et création d'une classe App avec une méthode main et l'exécuter

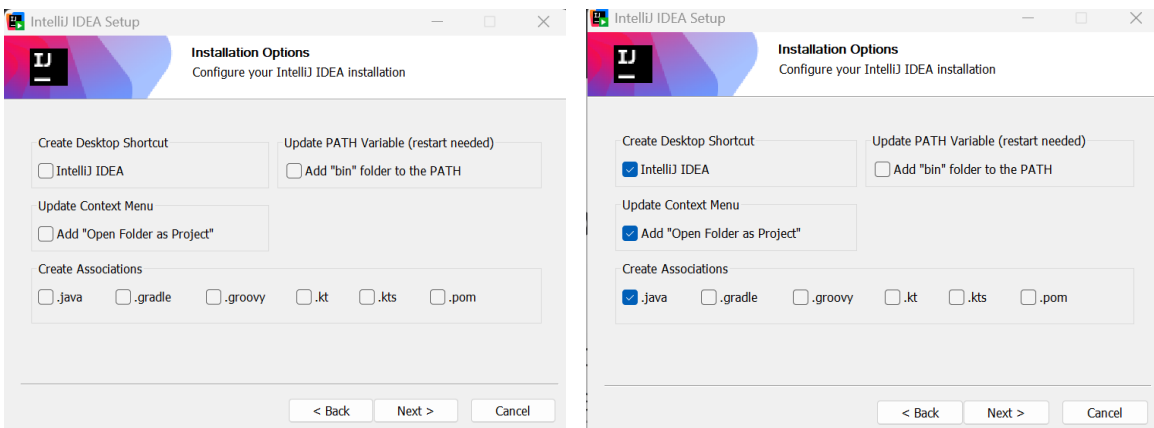
a- Installer / ouvrir IntelliJ IDEA Community

Télécharge l'exécutable et installe-le avec les paramètres par défaut :

<https://www.jetbrains.com/idea/download/?section=windows>



Coche ces éléments comme indiqué :



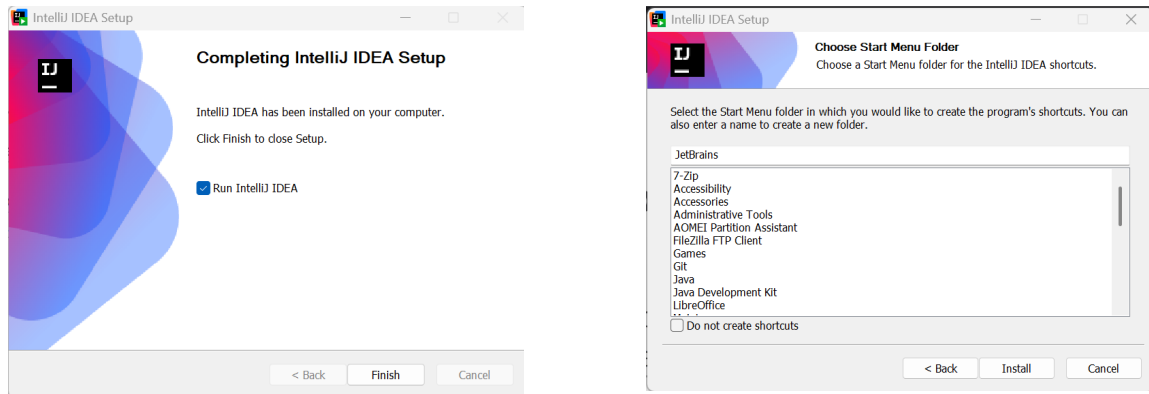
Laisse JetBrains par défaut et clique sur Install :

b- Ouvrir ton dépôt mycellius dans IntelliJ (sans casser Git)

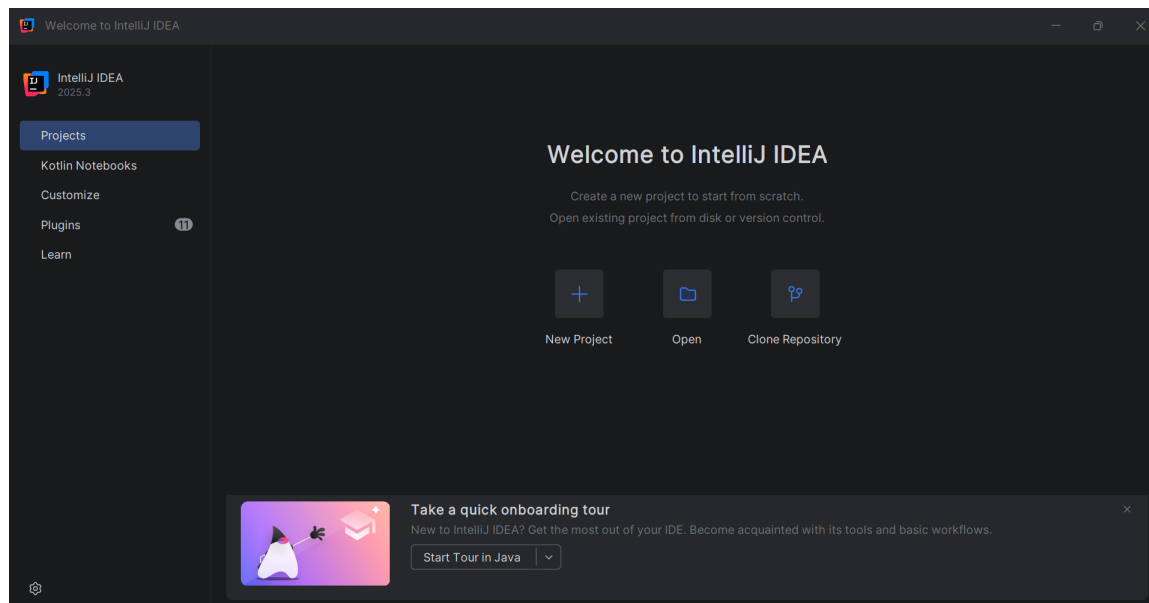
Au premier lancement, ouvre IntelliJ IDEA Community.

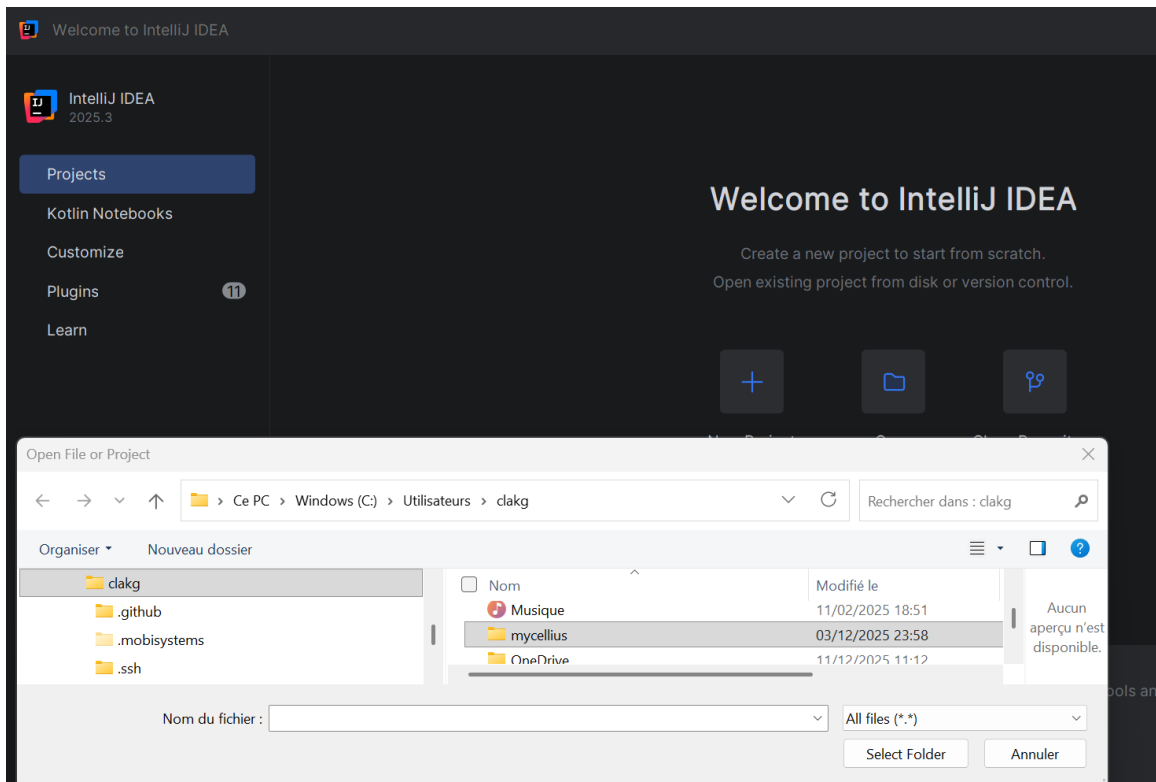
Si on te demande “**Import Settings ?**” réponds => **Do not import.**

Et après tu arrives sur l’écran d’accueil (New Project / Open / etc.). Oh yeahhhhhh ! Bon t'excites pas, on n' a rien fait encore là, ha ha haaaa !!!!

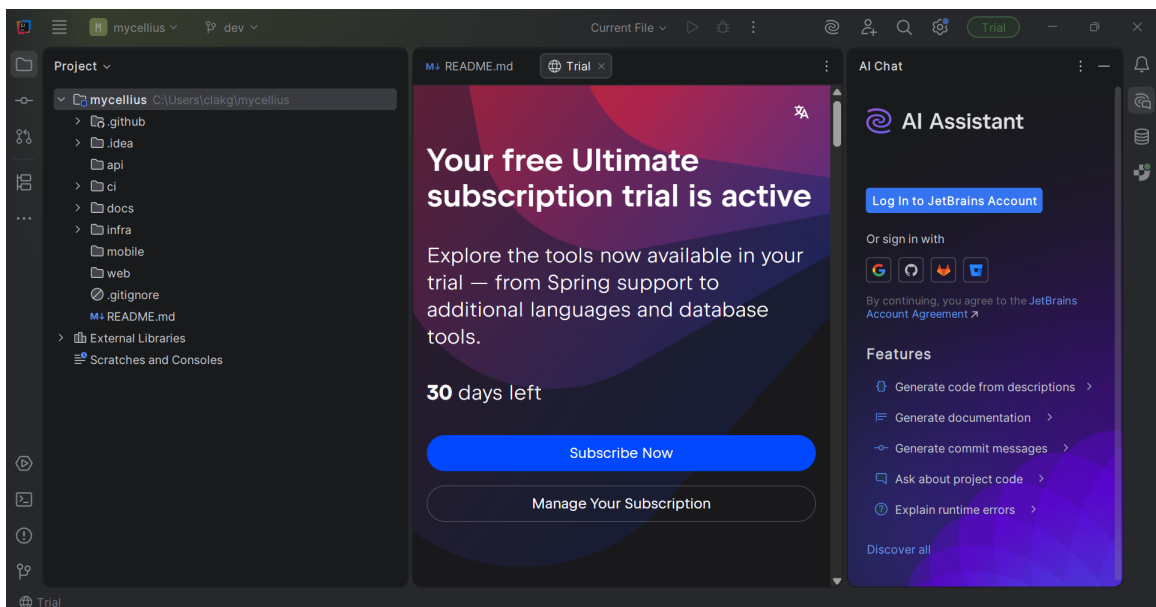


- clique sur **Open** > et trouve ton répertoire mycellius dans **C:\\Users\\ton-prenom**





Une fois que tu as cliqué sur le projet, tu as toute l'arborescence du projet créé au préalable sur
VsCode :



c- Créer le module Java api (Maven)

On va maintenant créer proprement le module Java api à l'intérieur de ce projet.

Mais avant, c'est quoi un "module Maven" dans notre cas ?

On a déjà un gros dossier mycellius qui contient tout le TP (docs, infra, etc.).

On va ajouter dedans un sous-dossier **api** qui sera notre projet Java :

- il aura sa structure **src/main/java**,
- un fichier **pom.xml** (configuration Maven),
- plus tard : on ajoutera nos classes Java (Tag, WikiPage, etc.).

Donc :

- mycellius = dépôt Git
- mycellius/api = projet Java (API)

On va maintenant créer le module api dans IntelliJ

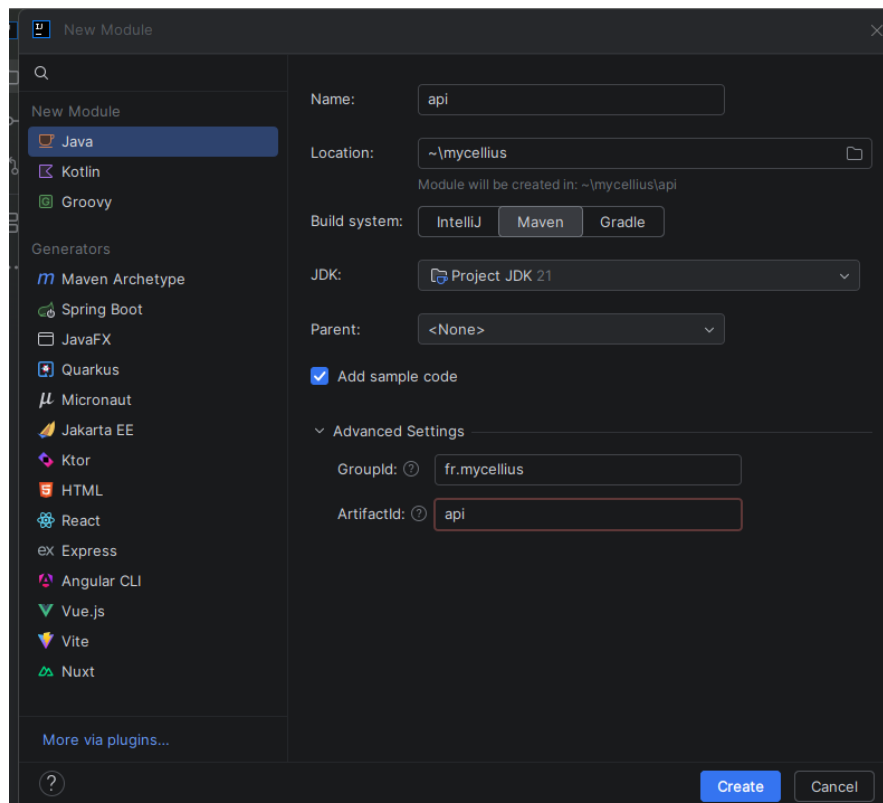
Tu es dans IntelliJ, avec le projet mycellius ouvert.

En haut, clique sur le menu File, puis sur **New → Module...**

Si tu vois plusieurs choses (Module / Module from Existing Sources), prends bien **Module...**

Une fenêtre "New Module" s'ouvre.

Dans la fenêtre "New Module" :



Sur cette fenêtre New Module, tu dois avoir :

Name : api

Location : ton dossier racine du projet.

Pour moi c'est => C:\Users\clakg\mycellius et en dessous la ligne grisée doit dire :

Module will be created in: ... \mycellius\api

Build system : Maven

JDK : Project JDK 21

Parent : <None> 

Add sample code :  coché

GroupId : fr.mycellius

ArtifactId : api

Puis clique sur **Create**.

Ci-dessous, retrouve les explications de l'utilité de chaque répertoire et fichier qui auront été généré :

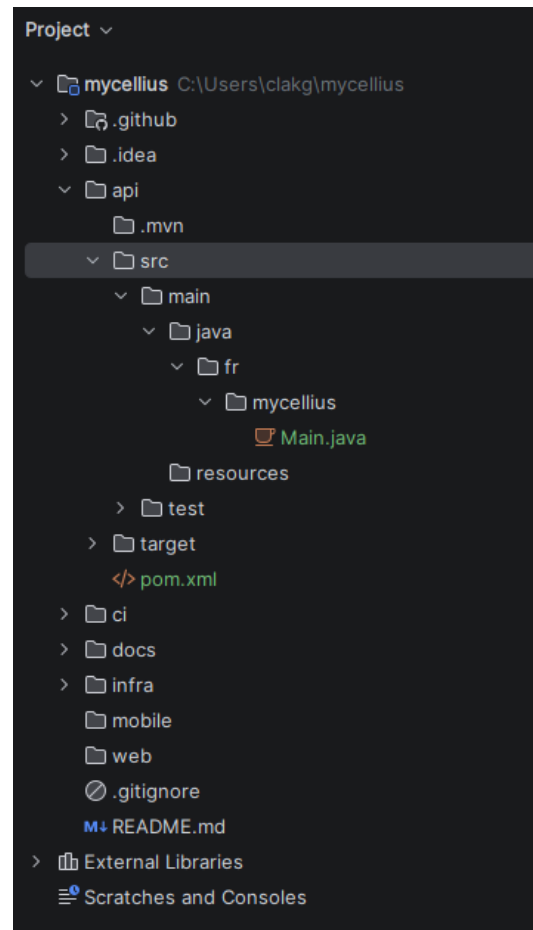
Élément	Rôle	Explication pédagogique
api/	Module Java Maven	C'est ton mini-projet Java. On pourra en avoir plusieurs plus tard (web, mobile, etc.)
pom.xml	Descriptif Maven	Contient les infos du projet (nom, version, dépendances...). C'est un peu la "carte d'identité" du module.
src/	"Source code"	Tous les fichiers source vont ici.
main/java/	Code principal	C'est là que tu mets le code qui sera compilé et exécuté.
fr/mycellius/	Package	Le package indique l'"adresse logique" de ta classe, comme un dossier virtuel. Il vient du GroupId défini tout à l'heure (fr.mycellius).
Main.java	Classe Java	C'est la première classe créée automatiquement — notre point d'entrée pour tester Java.

ATTENTION ! Quand vous allez Run le main, vous allez avoir un bug, et on va l'ignorer en supprimer ce Run tout simplement.

Faisons une vérification rapide de la structure (juste observer). Dans le panneau de gauche Project, tu as bien l'arborescence ci-contre :

Repères juste où est Main.java :

api/src/main/java/fr/mycellius/Main.java



Explication ligne par ligne du contenu de Main.java

Ouvre **Main.java**. Tu dois avoir quelque chose qui ressemble à ça (peut-être avec un for en plus, ce n'est pas grave) :

```
package fr.mycellius;

public class Main {
    public static void main(String[] args) {
        System.out.println("Hello and welcome!");
    }
}
```

Je t'explique tous les mots :

- **package fr.mycellius** :

- **package** : mot-clé qui indique dans quel "dossier logique" se trouve la classe.
- **fr.mycellius** : c'est le nom du package, souvent dérivé du nom de l'organisation/domaine.

Analogie :

Imagine que Java range les classes dans des boîtes. La boîte s'appelle fr.mycellius.

Plus tard on pourra avoir d'autres boîtes : fr.mycellius.domain, fr.mycellius.service, etc.

Ce package doit correspondre aux dossiers sur le disque :

> src/main/java/fr/mycellius/Main.java

- **public class Main** {

Découpe cette ligne :

- > **public** → signifie : "tout le monde peut voir cette classe". Ça veut dire qu'une autre classe, dans un autre package, pourra utiliser Main.
- > **class** → mot-clé pour dire : "je définis une classe", c'est-à-dire un plan de code.
- > **Main** → c'est le nom de la classe. En Java, par convention, *les noms de classes commencent par une majuscule*.
- > **{** → ouvre le bloc de code de la classe (tout ce qui est à l'intérieur "appartient" à la classe).

En gros, “Je déclare un plan de code public qui s’appelle Main. À l’intérieur, je mettrai des méthodes, dont la méthode spéciale Main.”

- **public static void main(String[] args) {**

C’est la ligne la plus “magique”.

C’est le point d’entrée du programme : c’est cette méthode que Java exécute quand tu fais “Run”.

Découpons :

- **public** : accessible par tout le monde (ici par la JVM elle-même).

- **static** veut dire que cette méthode appartient à la classe, pas à un objet particulier. Donc on n’a pas besoin de faire `new Main()` pour l’utiliser. La JVM peut appeler directement `Main.main(...)`.

- **void** → type de retour = rien.

La méthode exécute des instructions, mais ne renvoie pas de valeur.

- **main** → nom de la méthode. C’est un nom spécial : la JVM le cherche exactement écrit comme ça.

> `(String[] args)` → paramètre de la méthode.

> `String` = type “texte”.

> `[]` = tableau.

> `args` = nom de la variable.

C’est donc un tableau de chaînes de caractères, qui contient éventuellement les paramètres passés à ton programme en ligne de commande.

Pour l’instant, on ne s’en sert pas, mais la signature doit être exactement comme ça pour que Java reconnaisse `main`.

- **System.out.println("Hello and welcome!");**

Encore une découpe :

System → classe fournie par Java, qui regroupe des choses liées au système.

. → accès à un membre de la classe (comme dire `System` → “prends sa chose `out`”).

out → c’est un flux de sortie standard (Standard Output), en gros : la console.

****println**** → méthode qui affiche une ligne dans la console, puis passe à la ligne suivante. (printaffiche sans retour à la ligne,println` = “print line”).

("Hello and welcome!") → le texte qu’on veut afficher, entre guillemets " " = une String.

;
→ en Java, chaque instruction se termine par un point-virgule.

En gros, cette ligne indique : “Demande à System, via son flux out, d’imprimer cette ligne de texte dans la console.”

} de fin

Le } qui vient après la ligne System.out.println(...) → ferme la méthode main.

Le dernier } → ferme la classe Main.

Rappel... A quoi sert Maven :

Pour Java, on ne va pas compiler les fichiers à la main.

On va laisser Maven gérer ça pour nous.

Maven, c’est un outil qui impose une structure et sait aller chercher tout seul les bibliothèques dont on va avoir besoin (tests, Spring, etc.).

Tout est décrit dans un fichier pom.xml.

Comme ça, que ce soit sur mon PC, le vôtre ou GitHub, le projet se construit toujours de la même façon.”

Rappel... Java, Javac, JVM :

Quand tu écris du Java, il se passe 3 choses :

1. Tu écris un fichier Main.java par exemple
2. Le compilateur javac transforme ça en bytecode (fichier .class)
ex: Main.java => javac (compilateur) => Main.class
3. La JVM exécute ce bytecode et donne un résultat

En gros, quand on code en Java, on n’envoie pas directement le code au processeur.

On le compile en un langage intermédiaire (bytecode),

puis c’est la JVM, une machine virtuelle Java, qui l’exécute.

C’est pour ça qu’un programme Java peut tourner sur Windows, Linux, Mac

sans qu’on redéveloppe tout : seule la JVM change, le bytecode reste le même.

d- Découvrir JAVA et création d'une classe App avec une méthode main et l'exécuter

On va maintenant créer la **classe App** (sans encore toucher à Main) **qui déclare une classe publique nommée App**.

En Java, pour exécuter une application console, la JVM cherche une méthode très précise :

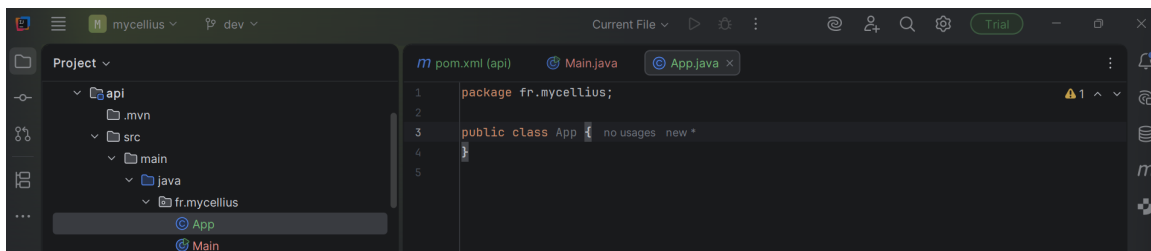
```
public static void main(String[] args)
```

C'est le "point d'entrée" du programme.

C'est un peu comme le "début" du script : tout commence là.

Créer le fichier App.java suivant dans IntelliJ :

api/src/main/java/fr/mycellius/**App.java**



package fr.mycellius : On dit à Java : "Cette classe App appartient au dossier logique fr.mycellius"
Ça correspond au chemin des dossiers : src/main/java/fr/mycellius/App.java.

public class App {

- > **public** : on autorise les autres parties du projet à utiliser cette classe.
- > **class** : on déclare une classe (un plan).
- > **App** : nom de la classe (convention : majuscule au début).
- > **{** : on ouvre le "bloc" de la classe → tout ce qui est dedans appartient à App puis on ferme le bloc de la classe avec **}** pour le final

Dans public class App {}, injecte :

```
public static void main(String[] args) {  
    System.out.println("Mycellius démarre !");  
    System.out.println("Bienvenue dans la séance 2 (bases Java).");  
}
```

Tu auras donc :

```
package fr.mycellius;

public class App {
    public static void main(String[] args) {
        System.out.println("Mycellius démarre !");
        System.out.println("Bienvenue dans la séance 2 (bases Java).");
    }
}
```

Output :

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program
Files\JetBrains\IntelliJ IDEA 2025.3\lib\idea_rt.jar=52544"
-Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8
-classpath C:\Users\clakg\mycellius\api\target\classes fr.mycellius.App
```

Mycellius démarre !

Bienvenue dans la séance 2 (bases Java).

Process finished with exit code 0

Ca veut dire que :

- la compilation a réussi,
- la JVM a lancé ta méthode main,
- tout s'est terminé proprement.

La dernière ligne :

Process finished with exit code 0

exit code 0 = tout va bien (pas d'erreur).

exit code ≠ 0 (1, 2, 130...) = souvent erreur ou arrêt inhabituel.

Donc, ton "Hello Mycellius" est validé

2. Bases de programmation Java : types, variables, opérations, contrôle

Objectif de la partie 2

À la fin de cette partie, tu seras capable de :

- déclarer des variables simples en Java
- comprendre les principaux types (int, double, boolean, String)
- faire des opérations arithmétiques de base
- utiliser des conditions (if/else) et une boucle for simple

2.1 Types et variables (int, double, boolean, String)

On va voir :

- Types primitifs : int, double, boolean, char.
- String (texte, pas primitif).
- Variables (boîtes avec un nom).
- Opérations simples (+, -, *, /, %).
- if / else pour des règles métier :
 exemple : si durée > 30 alors "ticket long".
- Boucle for ou while pour parcourir une liste plus tard (tu peux teaser Mycellius : "imagine on parcourt les pages d'un wiki").

Ce qu'on veut obtenir dans App.java

On part de la classe App existante dans api/src/main/java/fr/mycellius/App.java.

On veut que la méthode main affiche quelques informations simples à propos du projet Mycellius et manipule des variables.

Dans Main :

```
public static void main(String[] args) {  
    int nombreDePages = 0;  
    double tempsMoyenLecture = 3.5;  
    boolean wikiActif = true;  
    String nomProjet = "Mycellius";  
  
    System.out.println("Mycellius démarre !");  
    System.out.println("Bienvenue dans la séance 2 (bases Java).");  
    System.out.println("Nom du projet : " + nomProjet);  
    System.out.println("Nombre de pages : " + nombreDePages);  
    System.out.println("Temps moyen de lecture : " + tempsMoyenLecture + "  
minutes");  
    System.out.println("Wiki actif ? " + wikiActif);  
}
```

Dans IntelliJ:

- Ouvrir App.java.
- Repérer la méthode main (celle qu'on a déjà créée).
- Remplacer le contenu de main entre { et } par le code ci-dessus.
- Lancer le programme avec le bouton Run (triangle vert).
- Résultat attendu dans la console :

```
Mycellius démarre !  
Bienvenue dans la séance 2 (bases Java).  
Nom du projet : Mycellius  
Nombre de pages : 0  
Temps moyen de lecture : 3.5 minutes  
Wiki actif ? true
```

- Explication ligne par ligne (types + variables)

int nombreDePages = 0;

int : **type entier** (nombre sans virgule).

nombreDePages : nom de la variable. C'est le "nom de la boîte".

= 0 : on initialise cette variable à 0, c'est-à-dire que, pour l'instant, le wiki n'a aucune page.

Point de vue métier : on modélise un compteur de pages dans Mycellius.

Point de vue Java : on déclare une variable d'un type précis (int) et on lui donne une valeur initiale.

double tempsMoyenLecture = 3.5;

double : type **nombre à virgule**.

tempsMoyenLecture : nom de la variable.

= 3.5 : par exemple, on suppose qu'en moyenne une page prend 3,5 minutes à lire.

Pourquoi double ? Parce que 3,5 n'est pas un entier, donc **int** ne suffirait pas.

boolean wikiActif = true;

boolean : type **booléen**. Il ne peut valoir que true ou false.

wikiActif : nom de la variable.

= true : on dit que le wiki est "actif".

Booléen = idéal pour les drapeaux : "est actif ?", "est admin ?", "est loggé ?", etc.

String nomProjet = "Mycellius";

String : type texte (chaîne de caractères).

nomProjet : nom de la variable.

= "Mycellius" : la valeur est un texte entre guillemets.

Remarque : en Java, String est une classe (un objet), mais pour l'instant on l'utilise comme un type simple.

Les ***System.out.println*** avec +

System.out.println("Nom du projet : " + nomProjet);

"Nom du projet : " : c'est une chaîne de caractères littérale.

nomProjet : l'opérateur +, ici, sert à **concaténer** (coller) du texte.

Java transforme automatiquement nomProjet en texte et le joint au reste.

Donc, si nomProjet vaut "Mycellius", la ligne affichera : Nom du projet : Mycellius

Idem pour les autres lignes :

System.out.println("Nombre de pages : " + nombreDePages);

System.out.println("Temps moyen de lecture : " + tempsMoyenLecture + " minutes");

System.out.println("Wiki actif ? " + wikiActif);

=> On combine texte + variables, et Java produit une seule chaîne finale à afficher.

Rappel – Notion de variable

Une variable, c'est :

- un type (int, double, boolean, String...)
- un nom (nombreDePages, wikiActif...)
- une valeur (0, true, "Mycellius"...)

C'est exactement comme une boîte étiquetée :

- l'étiquette = le nom de la variable
- le type = ce que tu as le droit de mettre dedans
- la valeur = ce qui est réellement dans la boîte maintenant

2.2 Opérations sur les nombres et mise à jour de variables

On ajoute maintenant quelques opérations pour montrer comment les valeurs peuvent évoluer.

Ajoute ces lignes à la fin de la méthode main :

```
System.out.println("--- Mise à jour des statistiques ---");
nombreDePages = nombreDePages + 5;
tempsMoyenLecture = tempsMoyenLecture + 0.2;
System.out.println("Nouveau nombre de pages : " + nombreDePages);
System.out.println("Nouveau temps moyen de lecture : " + tempsMoyenLecture + "
minutes");
```

Explication ligne par ligne

System.out.println("--- Mise à jour des statistiques ---");

On affiche une séparation dans la console pour bien voir que ce qui suit est une "mise à jour".

`nombreDePages = nombreDePages + 5` => C'est une affectation.

À droite du `=` : `nombreDePages + 5` → on prend l'ancienne valeur (qui était 0) et on ajoute 5.

À gauche du `=` : `nombreDePages` → on réécrit la variable avec le nouveau résultat.

Si `nombreDePages` valait 0, il va valoir 5 après cette ligne.

En gros ici : "On ajoute 5 pages au wiki".

`tempsMoyenLecture = tempsMoyenLecture + 0.2` => Même mécanisme, mais sur un double.

On simule que la lecture moyenne a légèrement augmenté (3.5 → 3.7 par exemple).

System.out.println("Nouveau nombre de pages : " + nombreDePages);

System.out.println("Nouveau temps moyen de lecture : " + tempsMoyenLecture + " minutes");

On affiche les nouvelles valeurs pour vérifier qu'on a bien compris l'évolution.

Rappel – Opérations de base

```
int a = 10;
int b = 3;
int somme = a + b;
int diff = a - b;
int prod = a * b;
int div = a / b; // division entière : 10 / 3 = 3
int reste = a % b; // modulo : 1
```

2.3 Contrôle de flux : if / else (décision)

On veut maintenant que le programme commente la situation, par exemple :

- si le nombre de pages est 0 → le wiki est vide
- sinon → le wiki a du contenu

Ce qu’on ajoute dans main (à la fin) :

```
if (nombreDePages == 0) {
    System.out.println("Le wiki est vide pour l'instant.");
} else {
    System.out.println("Le wiki contient déjà des pages.");
}
```

Explication ligne par ligne

if (nombreDePages == 0) {

if : mot-clé qui signifie “si la condition est vraie alors exécuter ce bloc”.

() : parenthèses qui contiennent la condition à tester.

nombreDePages == 0 :

== signifie “égal à” (attention, un seul = sert à l’affectation).

donc on teste “est-ce que nombreDePages vaut 0 ?”.

Si la condition est vraie, Java exécute le bloc entre { et } juste en dessous.

System.out.println("Le wiki est vide pour l'instant.");

Ligne exécutée uniquement si nombreDePages vaut 0.

} else { => else : “sinon, si la condition est fausse”.

Dans notre cas : si nombreDePages n’est pas 0.

System.out.println("Le wiki contient déjà des pages.");

Cette ligne est exécutée si la condition de if est fausse.

Rappel – Schéma du if / else

“Si condition vraie → faire A
Sinon → faire B”.

2.4 Première boucle for simple

Objectif : montrer la répétition, par exemple pour simuler une progression.

Ce qu’on ajoute à la fin de main :

```
System.out.println("--- Simulation de chargement ---");  
for (int i = 1; i <= 3; i++) {  
    System.out.println("Chargement... étape " + i);  
}
```

Explication ligne par ligne

System.out.println("--- Simulation de chargement ---");

> Juste une ligne pour séparer visuellement.

for (int i = 1; i <= 3; i++) {

for : mot-clé pour une boucle.

int i = 1 : initialisation → i commence à 1.

i <= 3 : condition → la boucle continue tant que i est inférieur ou égal à 3.

i++ : après chaque tour, on ajoute 1 à i (c’est équivalent à i = i + 1).

=> Donc la boucle affichera 3 fois quelque chose, avec i = 1, puis 2, puis 3.

System.out.println("Chargement... étape " + i);

On affiche le numéro de l’étape à chaque tour.

3. POO de base : classe, objet, attribut, méthode

Avant de coder des classes métier Mycellius, on veut poser les notions POO sur un exemple très simple.

À la fin, tu seras en mesure de comprendre :

- classe = plan
- objet = instance concrète de ce plan
- attribut = données
- méthode = action / comportement
- constructeur = comment on crée proprement un objet
- private / public = protection des données (encapsulation)
- Encapsulation : private pour protéger les données, public pour exposer des méthodes propres.

3.1 Premier exemple POO simple : la classe Person (ou Utilisateur)

Ce qu'on veut obtenir c'est une classe Person qui représente une personne avec :

- un nom
- un âge
- une méthode qui sait se présenter

Création du fichier Person.java

Dans IntelliJ :

- > Clic droit sur le package fr.mycellius (dans api/src/main/java/fr/mycellius).
- > New → Java Class.
- > Nom de la classe : Person

IntelliJ génère un fichier Person.java avec quelque chose comme :

```
package fr.mycellius;  
  
    public class Person {  
  
    }  

```

On va le compléter.

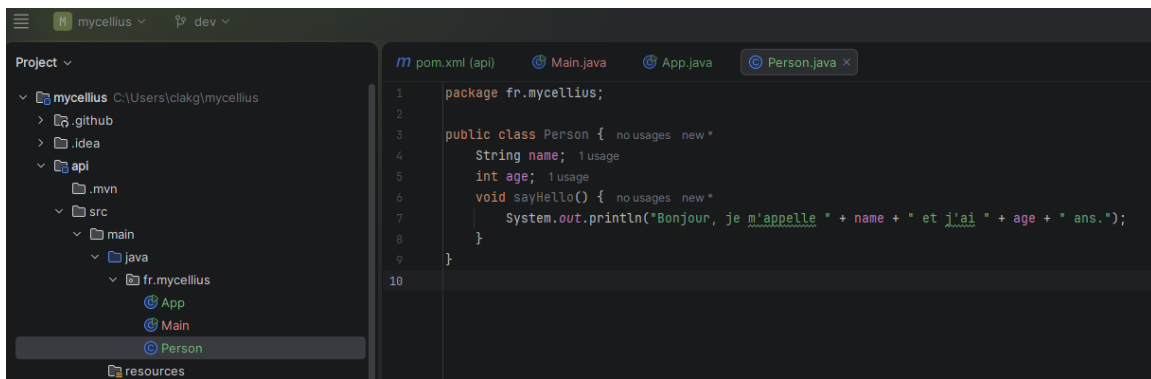
Code Person.java (version simple pour débiter) :

```
package fr.myclellius;

public class Person {

    String name;
    int age;

    void sayHello() {
        System.out.println("Bonjour, je m'appelle " + name + " et j'ai " +
            age + " ans.");
    }
}
```



Explication ligne par ligne

package fr.myclellius;

Comme pour App : indique que cette classe est rangée dans le package fr.myclellius.

public class Person {

public : accessible depuis d'autres classes.

class : mot-clé qui annonce une classe.

Person : nom de la classe (par convention, majuscule en premier).

{ : début du bloc de la classe.

String name;

String : type texte.

name : attribut de la classe Person.

En POO, on dit que name est un champ (field) ou attribut.

Il représente le nom de la personne.

int age;

int : type entier.

age : autre attribut, qui modélise l'âge de la personne.

void sayHello() {

void : la méthode ne renvoie rien.

sayHello : nom de la méthode.

() : pas de paramètre ici.

{ : début du bloc de la méthode.

System.out.println("Bonjour, je m'appelle " + name + " et j'ai " + age + " ans.");

On affiche un texte qui utilise les attributs name et age.

Le + sert à assembler le texte et les valeurs.

} => Fin de la méthode.

} => Fin de la classe.

3.2 Utiliser Person dans App.main

Dans App.main, après tes affichages actuels, ajoute :

```
System.out.println("--- Démo POO simple ---");
Person p = new Person();
p.name = "Alice";
p.age = 22;
p.sayHello();
```

Explication ligne par ligne

System.out.println("--- Démo POO simple ---");

Juste un séparateur visuel.

Person p = new Person();

Person : type de l'objet (classe).

p : nom de la variable qui va contenir l'objet.

new Person() : création d'un nouvel objet Person en mémoire.

On dit qu'on "instancie" Person.

=> En gros, tu fabriques une personne "vierge" qui a les attributs name et age, mais pas encore de valeurs.

p.name = "Alice";

On accède à l'attribut name de l'objet p.

On lui assigne la valeur "Alice".

On dit qu'on "remplit" l'objet.

p.age = 22;

Même chose pour l'âge.

p.sayHello();

On demande à l'objet p d'exécuter sa méthode sayHello.

La méthode va piocher dans name et age pour construire son message.

Limite de cette version

Tu peux faire p.age = -999; → ce n'est pas logique.

On n'a pas encore de garde-fou.

C'est un bon prétexte pour introduire ensuite :

- private
- constructeur
- getters.

3.3 Encapsulation (version plus propre)

On remplace Person par une version plus sérieuse, avec encapsulation.

Nouveau code Person.java :

```
package fr.mycellius;

public class Person {
    private final String name;
    private final int age;
    public Person(String name, int age) {
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Nom obligatoire");
        }
        if (age < 0) {
            throw new IllegalArgumentException("Âge invalide");
        }
        this.name = name.trim();
        this.age = age;
    }
    public String getName() {
        return name;
    }
    public int getAge() {
        return age;
    }
    public void sayHello() {
        System.out.println("Bonjour, je m'appelle " + name + " et j'ai " + age + " ans.");
    }
}
```

Explication des notions importantes

private final String name;

private : interdit l'accès direct depuis l'extérieur.

final : la valeur ne pourra plus être changée après la construction.

On protège les données de Person.

public Person(String name, int age) {

C'est un constructeur.

Il sert à construire un objet correctement initialisé.

On impose des règles dès la création.

if (name == null || name.trim().isEmpty()) {

On vérifie que name n'est pas null et pas vide.

Sinon → on refuse de créer une Person incohérente.

throw new IllegalArgumentException("Nom obligatoire");

throw : déclenche une exception → signale une erreur claire.

IllegalArgumentException : signifie "argument invalide pour cette méthode/constructeur".

this.name = name.trim();

this.name : l'attribut de l'objet.

name : le paramètre reçu.

On utilise this pour les distinguer.

On stocke une version nettoyée (trim).

getName / getAge

Getters : méthodes en lecture seule pour accéder aux attributs.

On ne donne pas de setters, donc Person est immuable (à âge fixe).

sayHello

Utilise les attributs internes, mais l'extérieur ne peut pas les casser.

Utilisation dans main (mise à jour)

Dans App.main, on remplace la précédente utilisation par :

```
System.out.println("--- Démo POO avec encapsulation ---");
Person bob = new Person("Bob", 30);
bob.sayHello();
System.out.println("Nom (via getter) : " + bob.getName());
```

On a maintenant :

- une vraie POO
- un constructeur qui applique des règles
- des données protégées (private)

=> Ce modèle est exactement ce qu'on va utiliser pour Mycellius (Tag, WikiPage, etc.).

4. POO appliquée à Mycellius : Tag & WikiPage

A ce stade, on va passer de l'exemple générique Person à des classes métier Mycellius :

- Tag : un mot-clé structuré
- WikiPage : une page du wiki avec id, titre, contenu

On applique les mêmes notions : private, final, constructeur, validation, getters.

4.1 Package domain

Pour organiser le code, on crée un package `fr.mycellius.domain`, qui contiendra nos classes métier.

Dans IntelliJ :

> Clic droit sur `fr.mycellius` → New → Package.

> Nom du package : `fr.mycellius.domain`.

IntelliJ va créer le dossier `domain` sous `fr/mycellius`.

4.2 Classe Tag (normalisation d'un mot-clé)

Ce qu'on veut :

- Un Tag qui contient une seule valeur (String)
- Cette valeur est nettoyée :
 - espaces autour supprimés
 - tout en minuscules
- On refuse un tag null ou vide.

Code Tag.java dans fr.mycellius.domain :

```
package fr.mycellius.domain;

import java.util.Locale;

public class Tag {
    private final String value;
    public Tag(String raw) {
        this.value = normalize(raw);
    }
    public static String normalize(String raw) {
        if (raw == null) {
            throw new IllegalArgumentException("tag null");
        }
        String v = raw.trim().toLowerCase(Locale.ROOT);
        if (v.isEmpty()) {
            throw new IllegalArgumentException("tag vide");
        }
        return v;
    }
    public String getValue() {
        return value;
    }
}
```

Explication ligne par ligne

package fr.mycellius.domain;

On place Tag dans le package métier domain.

import java.util.Locale;

Locale : utile pour toLowerCase(Locale.ROOT), afin d'avoir un comportement stable quelle que soit la langue du système.

public class Tag {

On déclare la classe Tag.

private final String value;

Attribut interne, privé, non modifiable après construction.

value : la version normalisée du tag.

public Tag(String raw) {

Constructeur qui prend la valeur brute (raw) saisie par l'utilisateur.

this.value = normalize(raw);

Au lieu de tout refaire ici, on délègue à une méthode normalize.

On centralise la règle de nettoyage.

public static String normalize(String raw) {

static : méthode liée à la classe, pas à un objet.

On peut appeler Tag.normalize(" Docker ") sans créer de Tag.

Retourne une String : la version propre du tag.

if (raw == null) {

On vérifie que l'entrée n'est pas null.

On préfère refuser tôt les données incohérentes.

throw new IllegalArgumentException("tag null");

On signale une erreur claire.

String v = raw.trim().toLowerCase(Locale.ROOT);

trim() : supprime les espaces au début et à la fin.

toLowerCase(Locale.ROOT) : met tout en minuscules, en utilisant un contexte neutre (ROOT) pour éviter les surprises liées à certaines langues.

if (v.isEmpty()) {

On interdit les tags vides (par exemple " " après trim).

throw new IllegalArgumentException("tag vide");

Même logique : erreur explicite.

return v;

On renvoie la version normalisée.

public String getValue() {

return value;

}

Getter pour lire la valeur interne, mais sans la modifier.

Explication métier

On veut éviter que dans la base de données on se retrouve avec 10 variantes de "Docker" :

"Docker", " docker ", "DOCKER"...

Avec Tag, tout devient "docker".

Plus tard, la recherche sera plus simple, et les tags seront cohérents.

4.3 Classe WikiPage (id + titre + contenu)

Ce qu'on veut :

– Une page avec :

- id : identifiant unique non vide
- title : obligatoire, propre
- content : texte, peut être vide mais pas null

– Un constructeur qui applique les règles

– Une méthode setTitle pour changer le titre en respectant la règle.

Code WikiPage.java dans fr.mycellius.domain :

```
package fr.mycellius.domain;

public class WikiPage {

    private final String id;
    private String title;
    private String content;

    public WikiPage(String id, String title, String content) {
        if (id == null || id.trim().isEmpty()) {
            throw new IllegalArgumentException("id obligatoire");
        }
        this.id = id.trim();
        setTitle(title);
        this.content = (content == null) ? "" : content;
    }

    public void setTitle(String title) {
        if (title == null || title.trim().isEmpty()) {
            throw new IllegalArgumentException("titre obligatoire");
        }
        this.title = title.trim();
    }

    public String getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public String getContent() {
        return content;
    }
}
```

Explication ligne par ligne

private final String id;

id : identifiant unique de la page.

final : on ne veut pas qu'il change après construction.

private String title;

Titre de la page, modifiable.

private String content;

Contenu de la page, modifiable.

public WikiPage(String id, String title, String content) {

Constructeur principal.

if (id == null || id.trim().isEmpty()) {

On vérifie que l'id n'est ni null ni vide (après trim).

|| signifie "ou".

throw new IllegalArgumentException("id obligatoire");

On refuse de créer une page sans id.

this.id = id.trim();

On stocke une version propre.

setTitle(title);

On réutilise la méthode setTitle pour appliquer la même règle de validation sur le titre.

Ça évite de dupliquer la logique.

this.content = (content == null) ? "" : content;

Opérateur ternaire : (condition) ? valeurSiVrai : valeurSiFaux.

Ici : si content est null, on stocke "", sinon on stocke content.

On évite d'avoir content = null, ce qui est plus confortable à l'usage.

public void setTitle(String title) {

Méthode pour modifier le titre.

```
if (title == null || title.trim().isEmpty()) {
```

Même règle : pas de titre vide ou null.

```
throw new IllegalArgumentException("titre obligatoire");
```

On refuse un titre invalide.

```
this.title = title.trim();
```

On stocke la version propre.

Getters (getId, getTitle, getContent)

Permettent de lire les champs sans les modifier directement.

Explication métier

On commence à voir une vraie “entité métier” : la page du wiki.

On a déjà des règles :

- pas de page sans id
- pas de page sans titre
- content toujours défini (au pire, vide)

Ce sont ces règles qui font la différence entre du “code qui compile” et une appli robuste.

4.4 Mini démo Mycellius dans App.main

Pour terminer la séance, on fait une petite démo qui utilise Tag et WikiPage dans main.

Dans App.main, après ce que tu as déjà, ajoute :

```
System.out.println("--- Démo Mycellius (Tag + WikiPage) ---");
```

```
Tag t1 = new Tag(" Docker ");
```

```
System.out.println("Tag normalisé : " + t1.getValue());
```

```
WikiPage page = new WikiPage("PAGE-001", "Installation Docker", "Étapes pour installer Docker sur Ubuntu.");
```

```
System.out.println("Page créée : " + page.getId() + " - " + page.getTitle());
```

Explication rapide

Tag t1 = new Tag(" Docker ");

On crée un tag avec une chaîne sale (espaces, majuscules).

t1.getValue() renverra "docker".

On démontre que normalisation fonctionne.

WikiPage page = new WikiPage("PAGE-001", "Installation Docker", "...");

On crée une page avec id, titre et contenu.

Puis on affiche id et titre.

Ce que vous aurez ont compris à la fin de la séance 2 (idéalement) :

- Comment fonctionne un programme Java simple (classe App, main, System.out.println).
- Ce qu'est un type, une variable, une opération, une condition, une boucle.
- Ce qu'est une classe, un objet, un attribut, une méthode, un constructeur.
- Ce que signifient public, private, final.
- Comment modéliser un vrai métier Mycellius avec Tag et WikiPage.

5. Tests/CI – partie Tests JUnit sur règles simples

On verra ici comment :

- ajouter JUnit à ton projet Maven api
- écrire des tests unitaires simples pour WikiPage et Tag
- lancer les tests en local (IntelliJ + Maven)
- comprendre le lien avec la future CI GitHub

5.1 Ajouter JUnit au projet Maven (api)

Important

Jusqu'ici, on a écrit du code "métier" (Tag, WikiPage...).

Maintenant on veut écrire du code qui TESTE ce code.

Pour ça, on utilise une bibliothèque de tests : JUnit 5 (aussi appelé JUnit Jupiter).

Et comme on est dans un projet Maven, c'est Maven qui va télécharger JUnit pour nous.

Étapes dans IntelliJ

1- Ouvrir le fichier pom.xml du module api. Dans la colonne de gauche (Project) :

- > développer api
- > double-cliquer sur pom.xml

2- Repérer la section <dependencies> et si tu vois déjà quelque chose comme :

```
<dependencies> ...
```

</dependencies> on va juste ajouter JUnit dedans.

S'il n'y a pas encore de balise <dependencies>, on va la créer.

3- Ajouter la dépendance JUnit

Dans pom.xml, assure-toi d'avoir au moins ceci (tu peux l'adapter à ton fichier, mais garde l'idée) :

```
<dependencies> <!-- autres dépendances éventuelles -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Explications rapides

- groupId / artifactId / version : “coordonnées” de la bibliothèque sur Internet
- scope test : JUnit ne sert qu’aux tests, pas au code de production
- Maven va télécharger tout seul JUnit quand on lancera mvn test.

4- Vérifier que Maven voit bien JUnit

- Dans IntelliJ, en haut à droite de la fenêtre du pom.xml, tu peux voir un bouton “Load Maven Changes” ou une petite icône d’éléphant / flèches.
- Clique dessus pour recharger le projet Maven.
- Si tout se passe bien, pas d’erreur rouge dans le pom.xml.

À partir de maintenant, on peut écrire des classes de test avec JUnit.

5.2 Créer le premier test : WikiPageTest

Objectif

Tester la classe WikiPage (dans fr.mycellius.domain) et vérifier que les règles suivantes sont bien appliquées :

- titre obligatoire (pas null, pas vide)
- titre nettoyé avec trim (“ SSH ” devient “SSH”)

5.2.1 Créer l’arborescence de test

Dans un projet Maven, le code de test est dans src/test/java.

On va donc créer :

```
api/src/test/java/fr/mycellius/domain/WikiPageTest.java
```

Étapes

Dans IntelliJ, dans le panneau Project :

- clic droit sur api
- New → Directory
- nom : src/test/java (si IntelliJ te propose déjà src/test/java, vérifie qu'il existe; sinon crée l'arborescence en plusieurs fois: src, puis test, puis java)

Puis dans src/test/java :

- clic droit sur java → New → Package
- nom : fr.mycellius.domain

Puis sur le package fr.mycellius.domain (dans src/test/java, attention à ne pas confondre avec celui de src/main/java) :

- clic droit → New → Java Class
- nom de la classe : WikiPageTest

IntelliJ génère un fichier comme ceci :

```
package fr.mycellius.domain;  
  
    public class WikiPageTest {  
  
    }  

```

5.2.2 Importer JUnit et écrire les premiers tests

On va écrire trois tests :

- testTitreNullInterdit
- testTitreVideInterdit
- testTitreNettoyeAvecTrim

Code complet proposé pour WikiPageTest.java :

```
package fr.mycellius.domain;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class WikiPageTest {
    @Test
    void testTitreNullInterdit() {
        assertThrows(IllegalArgumentException.class, () ->
            new WikiPage("PAGE-001", null, "contenu")
        );
    }
    @Test
    void testTitreVideInterdit() {
        assertThrows(IllegalArgumentException.class, () ->
            new WikiPage("PAGE-001", " ", "contenu")
        );
    }
    @Test
    void testTitreNettoyeAvecTrim() {
        WikiPage page = new WikiPage("PAGE-001", " SSH ", "contenu");
        assertEquals("SSH", page.getTitle());
    }
}
```

Explication des éléments importants

import org.junit.jupiter.api.Test;

On importe l'annotation `@Test` qui permet de marquer une méthode comme "méthode de test".

import static org.junit.jupiter.api.Assertions.*;

On importe les méthodes d'assertion (`assertThrows`, `assertEquals`...).

@Test au-dessus d'une méthode

Indique à JUnit : "cette méthode est un test à exécuter".

assertThrows(IllegalArgumentException.class, () -> new WikiPage(...));

On dit : "je m'attends à ce que ce code déclenche une `IllegalArgumentException`".

Si l'exception n'est pas levée → test en échec (rouge).

assertEquals("SSH", page.getTitle());

On vérifie que le titre stocké est bien "SSH" après trim.

5.2.3 Lancer les tests depuis IntelliJ

Dans `WikiPageTest.java` :

- clic droit dans le fichier → Run 'WikiPageTest'
- IntelliJ lance JUnit, exécute les trois méthodes `@Test` et affiche le résultat :
 - barre verte : tout est OK
 - barre rouge : au moins un test a échoué

Si un test échoue, lis le message d'erreur :

- soit la règle métier est mal codée,
 - soit le test est mal écrit.
- C'est exactement le cycle TDD (simplifié) : règle → test → correction.

5.3 Tests pour Tag

Objectif

Tester la classe `Tag` (`fr.mycellius.domain.Tag`) et vérifier les règles :

- tag normalisé en minuscules, espaces supprimés
- tag vide interdit
- tag null interdit

5.3.1 Créer TagTest

Dans `src/test/java/fr/mycellius/domain` :

- clic droit → New → Java Class
- nom : `TagTest`

Code proposé pour TagTest.java :

```
package fr.mycellius.domain;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class TagTest {
    @Test
    void testTagNormaliseEnMinusculesEtTrim() {
        Tag tag = new Tag(" Docker ");
        assertEquals("docker", tag.getValue());
    }
    @Test
    void testTagVideInterdit() {
        assertThrows(IllegalArgumentException.class, () ->
            new Tag(" "))
        );
    }
    @Test
    void testTagNullInterdit() {
        assertThrows(IllegalArgumentException.class, () ->
            new Tag(null)
        );
    }
}
```

Même logique que pour WikiPageTest :

- on teste les cas normaux (normalisation)
- on teste les cas d'erreur (null, vide)

En gros : un tag incohérent lève une exception claire, et les tests garantissent que ça reste vrai dans le temps.

5.4 Lancer tous les tests via Maven

IntelliJ sait lancer les tests, mais on veut aussi les lancer comme la CI le fera, c'est-à-dire via Maven.

Dans un terminal (VS Code ou IntelliJ) à la racine du repo mycellius :

```
mvn -f api/pom.xml test
```

Explication

mvn : lance Maven

-f api/pom.xml : dit à Maven "utilise ce pom.xml-là" (celui du module api)

test : lance la phase test = compilation + exécution des tests JUnit

Résultat attendu

Maven affiche un résumé avec "BUILD SUCCESS" si tout va bien

Si un test échoue, Maven affiche "BUILD FAILURE", et la CI sera en rouge

C'est EXACTEMENT la commande que GitHub Actions utilisera dans la CI.

Si tu as une erreur qui n'est pas citée, pas de panique :

- tape mvn -v

- si mvn n'est pas reconnu, on va l'installer.

- crée un répertoire C:\tools\maven\

- va sur : <https://maven.apache.org/download.cgi> et trouve "Binary zip archive", et télécharge apache-maven-3.9.x-bin.zip puis extrais le .zip et mets le répertoire dézippé dans le répertoire que maven que tu as créée. Tu dois obtenir : C:\tools\maven\apache-maven-3.9.11\ puis vérifie que tu as un dossier bin\

- Ouvre PowerShell (normal, pas besoin admin).

- Adapte le chemin ci-dessous à ta version exacte

```
$env:M2_HOME = "C:\tools\maven\apache-maven-3.9.9"
[Environment]::SetEnvironmentVariable("M2_HOME", $env:M2_HOME, "User")
```

- Ajoute le dossier bin de Maven au PATH utilisateur :

```
$oldPath = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path", "$oldPath;$env:M2_HOME\bin",
"User")
```

- **Ce qu'on vient de faire :**

M2_HOME = "où est installé Maven"

PATH = "liste des dossiers où Windows cherche les commandes"

→ on y a ajouté ...maven\bin pour que la commande mvn soit trouvée.

- reste sur powershell et refais les commandes :

```
mvn -v
```

```
cd C:\Users\clakg\mycellius
```

```
mvn -f api/pom.xml test
```

Explications

mvn → lance Maven

-f api/pom.xml → lui dit d'utiliser le pom.xml du module api

test → compile + exécute les tests JUnit

Et là normalement, tout est au vert !

```
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.9.3/junit-pl
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-commons/1.9.3/junit-pla
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running fr.mycellius.domain.TagTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.062 s -- in fr.mycellius.domain.TagTest
[INFO] Running fr.mycellius.domain.WikiPageTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in fr.mycellius.domain.WikiPageTest
[INFO] Results:
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.425 s
[INFO] Finished at: 2025-12-16T01:16:21+01:00
[INFO] -----
PS C:\Users\clakg\mycellius>
```

5.5 Ce que la CI fera plus tard (teaser)

Résumé :

En local : “j’écris du code, j’écris des tests, je lance mvn test ou le bouton Run Tests dans IntelliJ”

Sur GitHub : “la CI fera la même chose automatiquement à chaque push / PR”

6) CI – pipeline compile + test (≈ 30–40 min)

Objectif

- Transformer la CI “vide mais verte” de la séance 1 en une CI qui :
- compile l’API Java
- exécute les tests JUnit
- bloque les PR quand c’est rouge

6.1 Adapter le workflow ci.yml

On part du fichier existant :

`.github/workflows/ci.yml`

Tu peux soit :

- ajouter un job pour l’API Java
- soit enrichir un job existant

Ici je te propose un job dédié à l’API : `api-tests`.

Exemple de ci.yml complet ou de partie à intégrer (à adapter à ton fichier actuel) :

```
name: CI
on:
  push:
    branches: [ "dev", "test", "prod" ]
  pull_request:
    branches: [ "test", "prod" ]
jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Set up Java 21
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
      - name: Build and run tests for API
        run: mvn -f api/pom.xml -B test
```

Explication

on: push / pull_request

- La CI se déclenche sur :
 - > chaque push sur dev, test, prod
 - > chaque PR vers test ou prod

actions/checkout@v4

Récupère le contenu du repo pour que Maven ait accès au code.

actions/setup-java

Installe une JVM Java 21 sur la machine GitHub.

mvn -f api/pom.xml -B test

Lance la même commande que toi en local.

-B = mode batch (plus silencieux).

Résultat

Pour chaque push / PR :

- le module api est compilé,
 - les tests JUnit sont exécutés,
- si un test échoue → job en rouge → CI en rouge.

=> Vérifier que dans l'onglet "Actions" de GitHub, on voit le job api-tests et le détail de mvn test.

6.2 "Doc build ok" sur VM test (simplifié)

On garde les choses simples :

- CI verte = "doc build ok" symbolique
- on ne change pas encore le déploiement sur la VM test (page statique de la séance 1)

Informations :

Tant que les tests JUnit ne sont pas verts, on ne parle même pas de déploiement.

La qualité du code (tests OK) est un prérequis avant de pousser sur test / prod.

6.3 Résumé

Chemin complet :

j'écris une règle métier dans Tag ou WikiPage

j'écris un test JUnit qui vérifie la règle

en local : `mvn -f api/pom.xml test` → vert

je pousse sur dev → GitHub relance mvn test

si tout est vert → je peux faire une PR vers test

7) Workflow dev → PR test + “budget bugs”

Objectif

Reconnecter tout ce qu’on vient de voir (Java, JUnit, Maven, CI) avec le workflow Git dev/test/prod mis en place à la séance 1.

7.1 Workflow Git à réactiver

Sur ton poste (local) :

Se placer sur la branche dev
git checkout dev

Récupérer les dernières modifications (si besoin)

git pull origin dev

Coder dans api/

modifier ou ajouter des classes dans src/main/java/fr/mycellius/...
ajouter ou modifier des tests dans src/test/java/fr/mycellius/...

Lancer les tests localement (vivement conseillé)

```
mvn -f api/pom.xml test
```

Si c’est rouge, corriger avant de committer.

Préparer le commit

```
git status  
git add api/  
git commit -m "feat(java): add WikiPage and Tag tests"
```

Pousser vers dev

```
git push origin dev
```

Sur GitHub :

Aller dans l’onglet Actions
vérifier que le workflow CI s’exécute
s’assurer que le job api-tests est en vert

Créer une Pull Request dev → test

vérifier que la PR a un badge “checks” vert

Merger la PR vers test

IMPORTANT : On ne merge pas une PR si la CI est en rouge.

Les tests sont un filet de sécurité pour l'équipe.

7.2 "Budget bugs" – les erreurs classiques à anticiper

Ci-dessous une liste que tu peux reprendre quasiment telle quelle.

Erreurs Git

Je code sur la mauvaise branche (par exemple directement sur test).

Symptôme : git status montre une autre branche que dev.

Correction : git switch dev, puis éventuellement git cherry-pick ou recommencer proprement.

J'oublie de git add api/

Symptôme : la CI ne voit pas les nouveaux fichiers (tests manquants sur GitHub).

Correction : git add api/, git commit, git push.

Erreurs IDE / JDK

IntelliJ utilise le mauvais JDK (par exemple un vieux Java 8).

Symptôme : erreurs de compilation bizarres, ou version de langage limitée.

Correction : File → Project Structure → Project → Project SDK = 21, et pour le module api vérifier aussi le SDK.

Project SDK non configuré.

Symptôme : tout est souligné en rouge, alors que le code semble correct.

Correction : configurer un Project SDK (JDK 21) dans IntelliJ.

Erreurs Maven

JUnit non ajouté dans pom.xml.

Symptôme : les imports org.junit.jupiter... sont en rouge, ou mvn test ne trouve aucun test.

Correction : ajouter la dépendance JUnit dans <dependencies>.

mvn test échoue alors que "ça marche dans IntelliJ".

Symptôme : en local via IntelliJ tout est vert, mais la CI ou mvn test est rouge.

Causes possibles :

- tests ignorés dans IntelliJ (configuration de run incorrecte),
- dépendance manquante dans pom.xml (IntelliJ utilise un cache, Maven non),

- chemin de package différent entre `src/main/java` et `src/test/java`.

Erreurs d'import de JUnit

Mélange JUnit 4 / JUnit 5.

Symptôme : imports `org.junit.Test` au lieu de `org.junit.jupiter.api.Test`, ou méthodes d'assertion différentes.

Consigne :

utiliser systématiquement JUnit 5 pour ce cours,

donc : `import org.junit.jupiter.api.Test;` et `import static org.junit.jupiter.api.Assertions.*;`

Erreurs métier

On n'a pas reproduit les mêmes règles dans le test et dans la classe.

Exemple : le code accepte un titre vide, mais le test s'attend à une `IllegalArgumentException`.

C'est volontairement intéressant :

soit le code est faux → on le corrige,

soit le test est mal spécifié → on discute de la règle métier.

En présentant ce "budget bugs", on pose l'idée que l'erreur est normale, prévue, documentée.

Le but de la séance n'est pas de ne jamais se tromper, mais de savoir où regarder et comment relier :

- code Java

- tests JUnit

- Maven

- CI GitHub

- workflow Git (dev → test).